# VMMs and the Certifier

There are some side-channel attacks on VM's which employ adversarial VMMs to single step a VM and use this ability (along with the property that encrypted VMS generally encrypt DRAM in 16-byte blocks) to help discover in-memory keys. To avoid this, it is important to have a VMM that will not do this and check that the VMM is what you expect using TPM attested boot measurements. You also need to do this if you are on a machine that does not have Confidential Computing but supports a recursive attested boot of a VMM and VM image. Ideally, such a VMM is "minimalist" and avoids having device drivers.

An open source tool, Firecracker, can help. The repository is at https://firecracker-microvm.github.io. Firecracker is written in Rust that uses the Linux Kernel-based Virtual Machine (KVM) to create and manage lightweight virtual machines called microVMs and runs as user mode processes in the KVM. The user mode processes implement VMM functionality but rely on KVM for VMM support Firecracker was developed by Amazon Web Services.

The Firecracker site has information about reducing and hardening the KVM kernel image which is important. To be clear: Preventing single stepping requires a compliant KVM, hopefully minimized as suggested in the Firecracker doc. Firecracker itself doesn't block the attack. Firecracker does help make the VM interface safer by reducing the device interface.

Firecracker benefits include:

- Strong hardware-level security isolation of traditional VMs with the speed and low overhead of containers.
- Fast Boot Times, as little as 125 milliseconds.
- Low Memory Footprint: Consumes less than 5 MiB of memory overhead per microVM.
- Reduced Device Model: Strips out legacy and unnecessary emulated hardware (such as physical PCI buses, USB, BIOS, and ACPI tables) to minimize the attack surface and initialization time.
- Improved isolation using Security & Seccomp Jailer: It runs each microVM inside a strict process jailer using Linux seccomp filters to restrict system calls.
- Container Runtime support is integrated into projects like Kata Containers, Flintlock, and containerd as a secure alternative to standard Linux namespace/cgroup container isolation.

Here is more info from the Firecracker web site. A companion tool is AWS Lambda. Lambda MicroVMs is Firecracker is generally available on 64-bit Intel, AMD and Arm CPUs with support for hardware virtualization. The latest roadmap can be found here.

**Getting Started**

To get started with Firecracker, download the latest release binaries or build it from source.

You can build Firecracker on any Unix/Linux system that has Docker running (we use a development container) and bash installed, as follows:

```
git clone https://github.com/firecracker-microvm/firecracker
cd firecracker
tools/devtool build
toolchain="$(uname -m)-unknown-linux-musl"
```

The Firecracker binary will be placed at build/cargo_target/${toolchain}/debug/firecracker. For more information on building, testing, and running Firecracker, go to the quickstart guide.

The overall security of Firecracker microVMs, including the ability to meet the criteria for safe multi-tenant computing, depends on a well configured Linux host operating system. A configuration that we believe meets this bar is included in the production host setup document.

**Releases**

New Firecracker versions are released via the GitHub repository releases page, typically every two or three months. A history of changes is recorded in our changelog.

The Firecracker release policy is detailed here.

**Design**

Firecracker's overall architecture is described in the design document.

**Features & Capabilities**

Firecracker consists of a single micro Virtual Machine Manager process that exposes an API endpoint to the host once started. The API is specified in OpenAPI format. Read more about it in the API docs.

The API endpoint can be used to:

1. Configure the microvm by:
2. Setting the number of vCPUs (the default is 1).
3. Setting the memory size (the default is 128 MiB).
4. Configuring a CPU template.
5. Add one or more network interfaces to the microVM.
6. Add one or more read-write or read-only disks to the microVM, each represented by a file-backed block device.
7. Trigger a block device re-scan while the guest is running. This enables the guest OS to pick up size changes to the block device's backing file.
8. Change the backing file for a block device, before or after the guest boots.
9. Configure rate limiters for virtio devices which can limit the bandwidth, operations per second, or both.
10. Configure the logging and metric system.
11. [BETA] Configure the data tree of the guest-facing metadata service. The service is only available to the guest if this resource is configured.
12. Add a vsock socket to the microVM.
13. Add a entropy device to the microVM.
14. Add a pmem device to the microVM.
15. Configure and manage memory hotplugging.

16. [Developer Preview] Hot-plug and hot-unplug virtio PCI devices while the VM is running.
17. Start the microVM using a given kernel image, root file system, and boot arguments.
18. [x86_64 only] Stop the microVM.
19. Demand fault paging and CPU oversubscription is enabled by default.
20. Advanced, thread-specific seccomp filters for enhanced security.
21. Jailer process for starting Firecracker in production scenarios; applies a cgroup/namespace isolation barrier and then drops privileges.

**Known issues and Limitations**

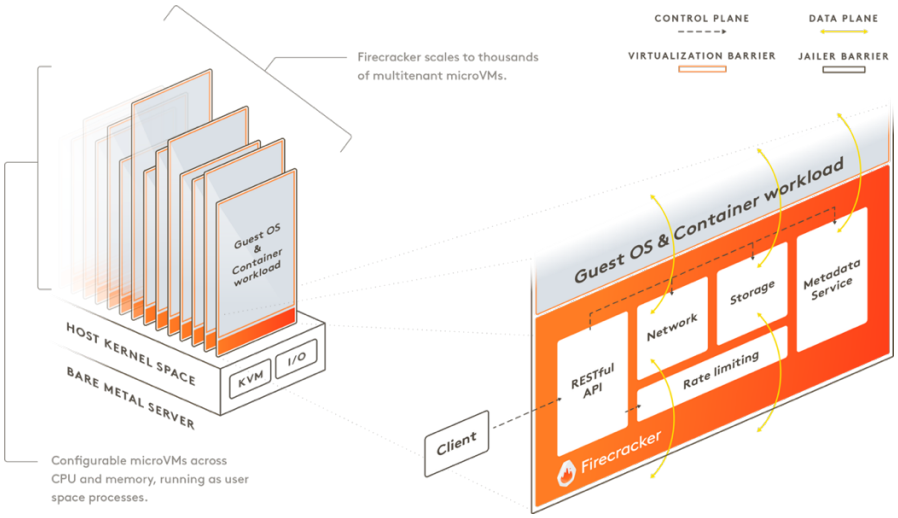
The pl031 RTC device on aarch64 does not support interrupts, so guest programs which use an RTC alarm (e.g. hwclock) will not work.

**Performance**

Firecracker's performance characteristics are listed as part of the specification documentation. All specifications are a part of our commitment to supporting container and function workloads in serverless operational models, and are therefore enforced via continuous integration testing.

## How It Works

The following diagram depicts an example host running Firecracker microVMs.



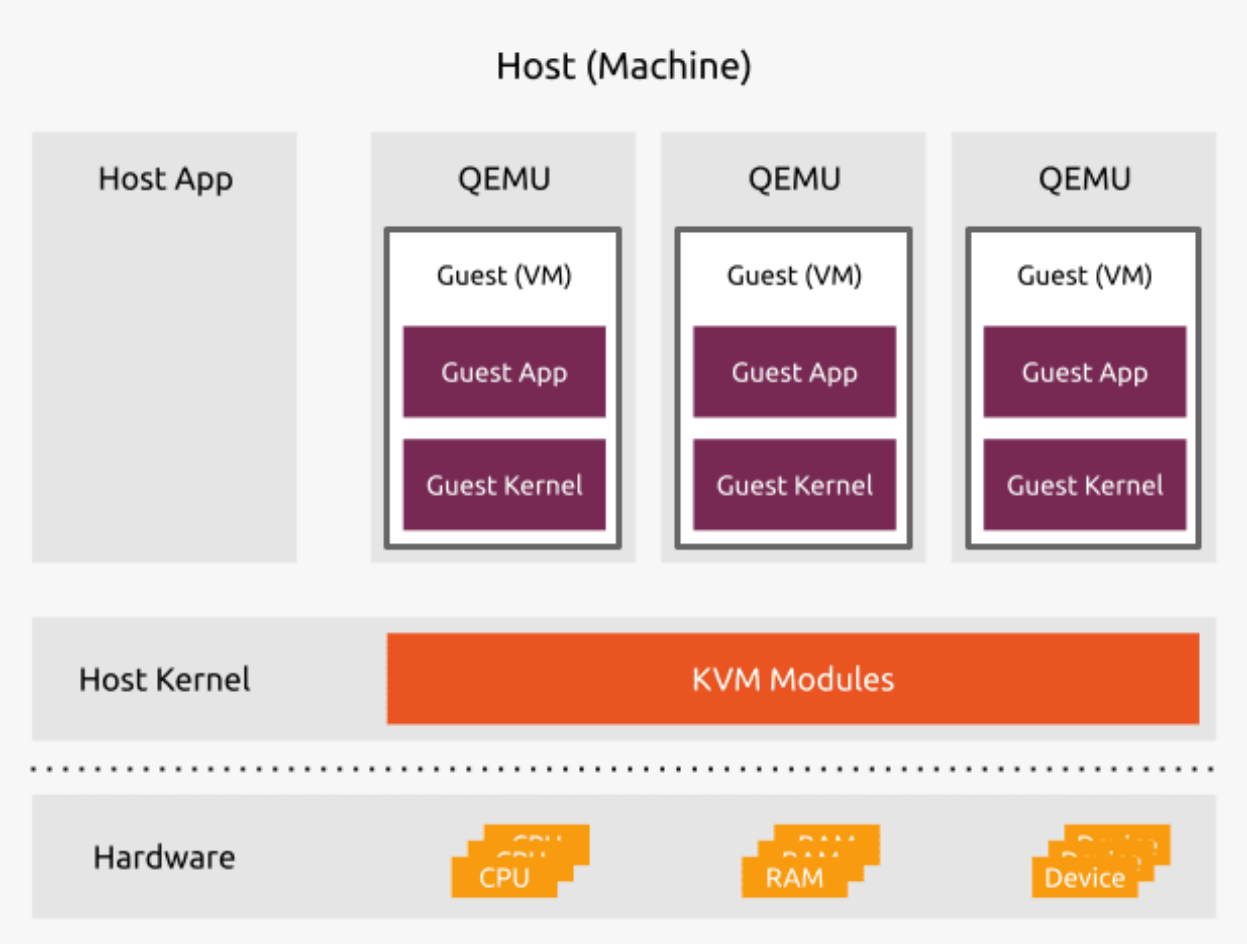

Firecracker runs in user space and uses the Linux Kernel-based Virtual Machine (KVM) to create microVMs. The fast startup time and low memory overhead of each microVM enables you to pack thousands of microVMs onto the same machine. This means that every function, container, or container group can be encapsulated with a virtual machine barrier, enabling workloads from different customers to run on the same machine, without any tradeoffs to security or efficiency. Firecracker is an alternative to QEMU , an established VMM with a general purpose and broad feature set that allows it to host a variety of guest operating systems.

You can control the Firecracker process via a RESTful API that enables common actions such as configuring the number of vCPUs or starting the machine. It provides built-in rate limiters, which allows you to granularly control network and storage resources used by thousands of microVMs on the same machine. You can create and configure rate limiters via the Firecracker API and define flexible rate limiters that support bursts or specific bandwidth/operations limitations. Firecracker also provides a metadata service that securely shares configuration information between the host and guest operating system. You can set up and configure the metadata service using the Firecracker API. Each Firecracker micro VM is further isolated with common Linux user-space security barriers by a companion program called "jailer". The jailer provides a second line of defense in case the virtualization barrier is ever compromised.

## Structure of KVM support

### What is a KVM hypervisor?

KVM hypervisor enables full virtualisation capabilities. It provides each VM with all typical services of the physical system, including virtual BIOS (basic input/output system) and virtual hardware, such as processor, memory, storage, network cards, etc. As a result, every VM completely simulates a physical machine.




KVM is available as a Linux kernel module. It plugs directly into the kernel's code and allows it to function as a hypervisor. Every VM runs as a separate Linux process under systemd, with dedicated virtual hardware resources attached. KVM can only be used on a processor with hardware virtualisation extensions, such as Intel-VT or AMD-V.

## Containerization and KVM

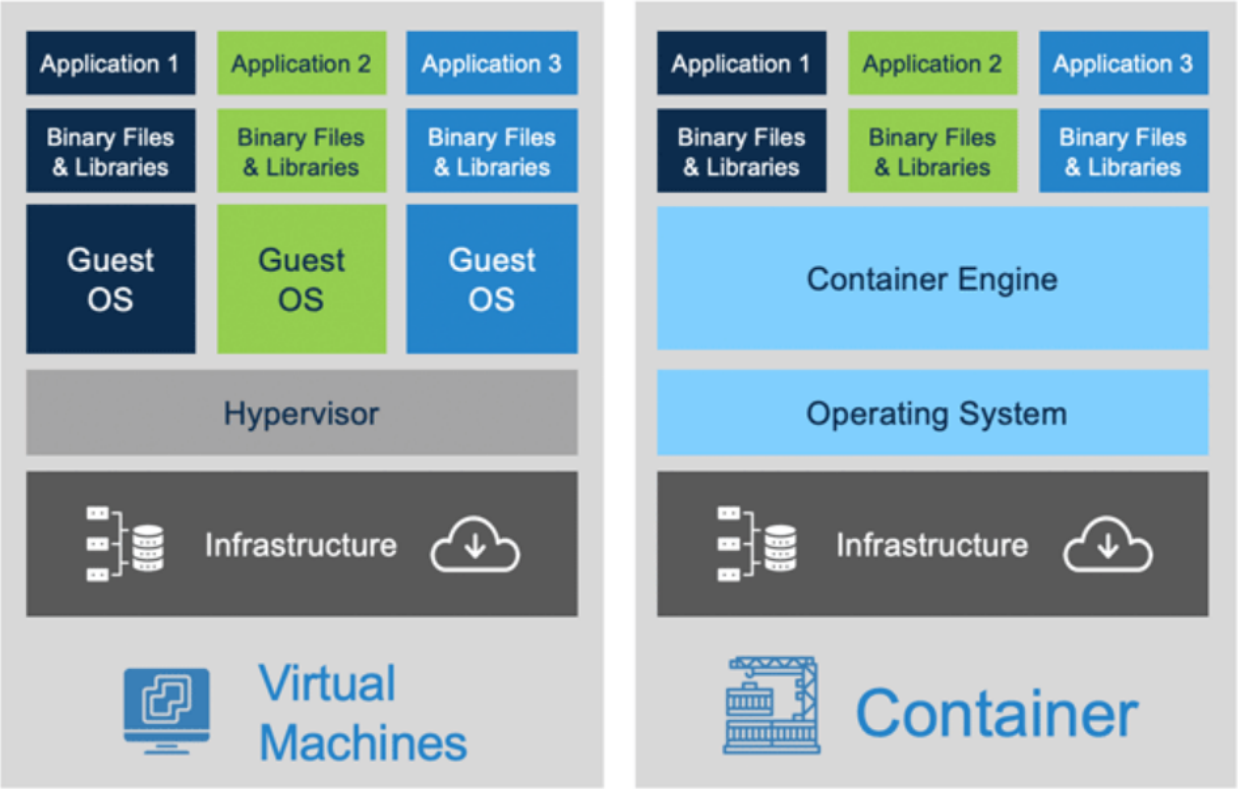
**KVM (Hardware Virtualization):** Turns the Linux kernel into a hypervisor. Each Virtual Machine (VM) runs its own complete guest operating system (Linux, Windows, BSD, etc.), has its own kernel, and uses emulated or passed-through virtual hardware. [1, 2]

**Containerization (OS-Level Virtualization):** Uses Linux kernel features like *namespaces* and *cgroups* to isolate application processes. All containers share a single host operating system kernel. [1, 2, 3]

## Feature Comparison

| Feature | KVM (Virtual Machine) | Containerization (e.g., Docker, LXC) |
|---|---|---|
| **Virtualization Level** | Full hardware virtualization | OS-level application isolation |
| **Kernel** | Separate kernel per instance | Shared host kernel |
| **Guest OS Support** | Linux, Windows, BSD, macOS | Linux only natively (Windows/macOS require a VM layer) |
| **Resource Footprint** | Heavier (512 MB to 4 GB+ RAM typical) | Lightweight (10 to 200 MB RAM typical) |
| **Boot / Startup Time** | Seconds to minutes | Milliseconds to seconds |
| **Isolation Strength** | Strong (complete hardware/OS boundary) | Moderate (shared kernel vulnerability risk) |
| **Workload Density** | Lower density per host | High density (up to 5x more instances on same RAM) |

You can do both!


Application 1
Application 2
Application 3
Binary Files & Libraries
Binary Files & Libraries
Binary Files & Libraries
Guest OS
Guest OS
Guest OS
Hypervisor
Infrastructure
Virtual Machines
Application 1
Application 2
Application 3
Binary Files & Libraries
Binary Files & Libraries
Binary Files & Libraries
Container Engine
Operating System
Infrastructure
Container